CODE STYLING AND CONSISTENCY

COMPILED BY: Najaaaz Nabhan & Aamir Mohamed Thahir Ali

CONTENTS

CONTENTS **1**
COMMENTS IN UNITY C# **2**
Commonly Used Tags In Header Of Classes Or Methods 2
Inline Comments 4
Maintaining Function Documentation 5
Conclusion In Comments 5
COMMENTS IN GODOT (GDSCRIPT) **6**
Commonly Used Tags in Class or Function Headers 6
Inline Comments 8
Maintaining Function Documentation 8
Conclusion on Comments 9
BEST PRACTICES FOR METHODS & VARIABLES **10**
Best Practices In Methods 10
Best Practices In Variables **11**
GITHUB COMMITTING STANDARD **13**
Git Commits 13
Git Branch 15
Git Pull Requests 16

COMMENTS IN UNITY C#

Consistency is very important in any code base and makes it more approachable and understandable for all members of the team and even others who read it (Eg: employers who see through your GitHub)

There isn't a strict standard for commenting in C# for Unity and each developer may vary their style.

Project Pixel has decided to use the commonly used documentation style called **"XML Documentation"**

Commonly Used Tags In Header Of Classes Or Methods

1. **<summary>**
   The *<summary>* tag provides a brief description of what a class or method does.

```
/// <summary>
/// Represents a player in the game.
/// </summary>
public class Player
{
  // Class implementation
}
```

2. **<returns>**
   The *<returns>* tag is used to describe the return value of a method.

```
/// <summary>
/// Adds two integers and returns the result.
/// </summary>
/// <returns>The sum of the two integers.</returns>
public int Add(int a, int b)
{
  return a + b;
}
```

3. **<param>**
   The *<param>* tag is used to describe parameters of a method.

```
/// <summary>
/// Calculates the area of a rectangle.
```

```
/// </summary>
/// <param name="length">The length of the rectangle.</param>
/// <param name="width">The width of the rectangle.</param>
/// <returns>The area of the rectangle.</returns>
public double CalculateArea(double length, double width)
{
  return length * width;
}
```

4. **<todo>**
   Use the *<todo>* tag to indicate that a particular feature or improvement is your to-do list. Provide a brief description of what needs to be done.

```
/// <summary>
/// Processes user orders.
/// </summary>
/// <todo>
/// Implement support for order cancellations.
/// </todo>
public OrderStatus ProcessOrder(Order order)
{
  // Implementation
}
```

5. **<bug>**
   Use the *<bug>* tag to highlight a known issue or bug in your code. Include details about the bug.
   If possible, provide information on any workarounds or temporary solutions.
   Committing changes with a known bug is discouraged, and should only be considered as an option of last resort.

```
/// <summary>
/// Divides two numbers.
/// </summary>
/// <bug>
/// There is a known issue where division by zero is not handled properly.
/// </bug>
public double Divide(int numerator, int denominator)
{
  // Implementation
}
```

Inline Comments

1. **Clarifying Code Logic:**
   Inline comments serve to explain complex or non-intuitive sections of code. They act as concise notes to help developers quickly grasp the logic and functionality of **specific lines or blocks of code.**

```
// Increment the player's score
score += 10;
```

2. **Providing Context for Unity-Specific Code:**
   Unity projects often involve specific concepts like game objects, components, and scenes. Inline comments can provide context to Unity-specific code, especially for employers who aren't familiar with Unity

```
// Check if the enemy is within the player's detection range
if (Vector3.Distance(enemy.position, player.position) < playerRange) {
}
```

Maintaining Function Documentation

In addition to documenting what each function does, it's beneficial to document *who maintains it*.
This can be achieved by adding a **<remarks>**
This is especially useful since it makes everyone aware of who made the function and know who to inform in terms of refactoring it, fixing bugs etc.
This comment MUST be used above all classes and methods

```
/// <summary>
/// Represents a player in the game.
/// </summary>
/// <remarks>
/// Maintained by: Najaaz Nabhan
/// Refactored by: Antosh Nikolak
/// </remarks>
public class Player
{
  // Class implementation
}
```

Conclusion In Comments

Overall XML documentation is a vast subject and takes time to get used to it.

It might seem a bit overwhelming at first, but with time, you'll get the hang of it. I've covered the main tags we'll use for now, and as our projects evolve, I might add more details.
If you notice anything missing or confusing about commenting, just let your team lead or Najaaz know.

Quick reminders:

1. **Different Comment Styles:**
   Use /// with 3 slashes for XML documentation and // with 2 slashes for inline comments to keep things neat.

2. **Tag Basics:**
   XML tags start with < and end with >, so keep an eye on the formatting

3. **Inline Comment Moderation:**
   Only use inline comments when really needed to maintain a clean and easy-to-read code.

COMMENTS IN GODOT (GDSCRIPT)

Consistency is essential in any codebase, helping make it approachable and understandable for all team members and others who may view it (e.g., prospective employers on GitHub).

GDScript doesn't enforce strict commenting standards, so styles can vary. Project Pixel will follow a structured documentation style similar to **Python docstrings** to maintain consistency.

Commonly Used Tags in Class or Function Headers

1. **@brief**

The `@brief` tag provides a quick overview of what a class or function does

```
## @brief Represents a player in the game.
class_name Player

## Class implementation
```

2. **@return**
   Use the `@return` tag to describe the return value of a function.

```
## @brief Adds two integers and returns the result.[br]
## @return The sum of the two integers.
func add(a: int, b: int) -> int:
    return a + b
```

3. **@param**
   The @param tag describes the parameters of a function.

```
## @brief Calculates the area of a rectangle. [br]
## @param length The length of the rectangle. [br]
## @param width The width of the rectangle. [br]
## @return The area of the rectangle.
func calculate_area(length: float, width: float) -> float:
    return length * width
```

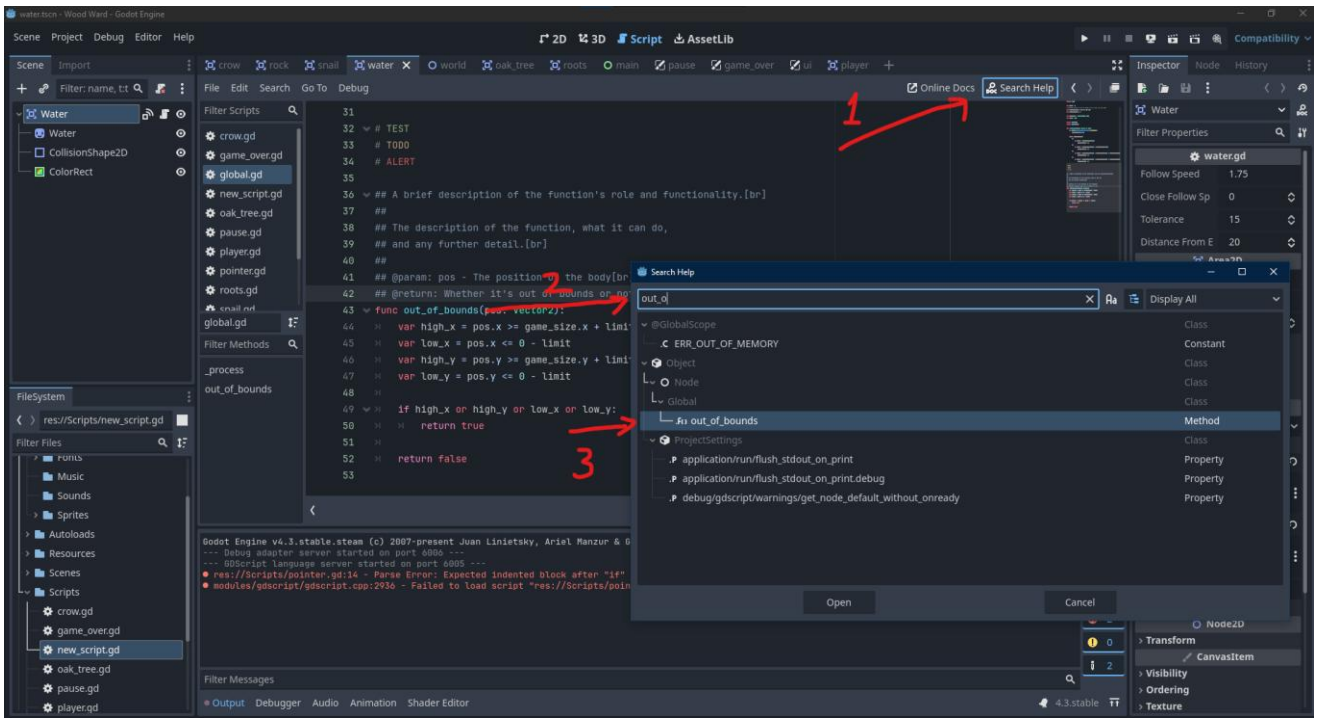
4. **@todo**
   Use the @todo tag to note features or improvements on your to-do list with a brief description.

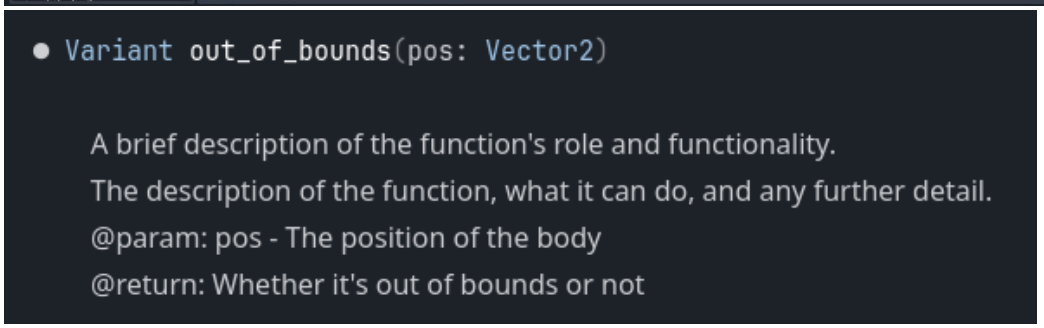
```
## @brief Processes user orders. [br]
## @todo Add support for order cancellations.
func process_order(order):
    pass # Implementation
```

5. **@bug**
   Highlight known issues or bugs with @bug. Mention details, workarounds, or temporary solutions, if any. Commit known bugs sparingly and only as a last resort.

```
## @brief Divides two numbers. [br]
## @bug Division by zero is not handled properly.
func divide(numerator: int, denominator: int) -> float:
    return numerator / denominator
```

***Note: for every class or function documentation we add [br] at the end to show a line break. This helps Godot inbuilt documentation builder to understand it better**
See the example below:





Inline Comments

1. **Clarifying Code Logic**
   Inline comments can clarify complex or non-intuitive sections of code. These concise notes help developers grasp specific lines or blocks quickly.

```
## Increment the player's score
score += 10
```

2. **Providing Context for Godot-Specific Code**
   Godot's codebase often involves unique concepts such as nodes, signals, and scenes. Inline comments can provide helpful context for code, especially for those unfamiliar with Godot

```
## Check if the enemy is within the player's detection range
if enemy.global_position.distance_to(player.global_position) < player_range:
    pass
```

Maintaining Function Documentation

To enhance maintainability, add a @remarks tag to indicate who maintains each function. This helps keep track of ownership for refactoring, bug fixes, etc.

```
## @brief Represents a player in the game.
## @remarks Maintained by: Najaaz Nabhan
## Refactored by: Antosh Nikolak
class_name Player

# Class implementation
```

Conclusion on Comments

Docstring-style commenting in GDScript may feel overwhelming at first, but it quickly becomes intuitive. This guide covers the primary tags we'll use, and more may be added as our projects evolve.

Quick Reminders:

- **Different Comment Styles**: Use `#` for both inline and docstring-style comments to maintain neatness.
- **Tag Basics**: Tags start with `@` and a descriptive keyword to maintain consistency.
- **Inline Comment Moderation**: Use inline comments only when truly necessary to keep code clean and readable.

If you have questions or suggestions on improving the commenting standards, reach out to your team lead or Najaaz

BEST PRACTICES FOR METHODS & VARIABLES

Best practices for writing functions and methods are essential for producing clean, maintainable, and efficient code.

Best Practices In Methods

1. **Use Pascal Casing for class and method names**
   Use Pascal Case (capitalising the first letter of each word) for method names. For example, *CalculateArea()* or *PrintMessage()*, etc.

2. **Descriptive Names**
   Choose descriptive and meaningful names that convey the purpose of the method. Avoid overly abbreviated or cryptic names.

3. **Avoid using underscores**
   In C#, it's common to use Pascal Case rather than underscores for method names. For example, **use** *CalculateArea()* over *calculate_area()*.

4. **Keep the methods short and sweet**
   - It's highly recommended to reuse code and keep your functions short.
   - Try and keep your functions less than 30 lines long
   - We highly recommend you follow the DRY principle (“Don't repeat yourself”)
   - The main idea behind the DRY principle is that each piece of knowledge or logic in a system should only exist in a single place.

```
// Without DRY Principle
static double CalculateCircleArea(double radius) {
    double pi = 3.14159;
    return pi * radius * radius;
}

static double CalculateRectangleArea(double length, double width) {
    return length * width;
}

// With the DRY Principle
static double CalculateArea(string shape, params double[] args) {
    if (shape == "circle") {
        double radius = args[0];
        return CalculateCircleArea(radius);
    } else if (shape == "rectangle") {
        double length = args[0];
        double width = args[1];
        return CalculateRectangleArea(length, width);
    } else {
        throw new ArgumentException("Invalid shape specified");
```

```
    }
}
```

Best Practices In Variables

1. **Use camelCase for local and global variables**
   For example use call your variables *"fooBar"* instead of *"foo_bar"*, to keep consistency

2. **Use uppercase for constants**
   Declare variables in the top of the script with UPPERCASE variable names for constants

```
const int MAX_HEALTH = 100;
```

2. **Use descriptive names**
   Choose descriptive names

3. **Avoid magic numbers**
   If you have a multiplier that doesn't make sense on its own, you can use this pattern below to make your code self documenting/

```
// Magic Number
int result = someValue * 42;

// Named Constant
const int Multiplier = 42;
int result = someValue * Multiplier;
```

4. **Use 'var' appropriately**

```
// Good
var employeeName = "John Doe";
var numbers = new List<int>();
// Avoid
List<int> numbers = new List<int>();
```

GITHUB COMMITTING STANDARD

Following a **consistent standard** when it comes to code commits and interactions within your team's GitHub repository is vital. These practices ensure consistency, improve collaboration, enhance code quality, and contribute to the overall success of your game development project.

Git Commits

It's always a good practice to commit your code changes frequently. As you may be aware, each commit requires a commit message.
In the following section, we'll explore best practices that you should follow when crafting your **commit messages**.

1. **Present Tense**
   Write commit messages in the present tense. This creates a sense of immediacy and makes the message more actionable
   (e.g., "*Fix bug*" instead of "*Fixed bug*").

2. **Use a Prefix**
   Consider using prefixes to categorise commits. Common prefixes include:

   a. **feat**: for new features or functionalities

```
git commit -m "[feat]: Add jumping mechanic"
```

   b. **fix:** for bug fixes

```
git commit -m "[fix]: Resolve collision detection issue"
```

   c. **refactor**: for code refactoring

```
git commit -m "[refactor]: Improve code readability"
```

   d. **docs:** for documentation updates

```
git commit -m "[docs]: Update README"
```

   e. **style:** for code style changes

```
git commit -m "[style]: Format code "
```

   f. **test:** for adding or modifying tests

```
git commit -m "[test]: Add unit tests for character controller"
```

   g. **chore:** for routine tasks, maintenance, or non-code changes

```
git commit -m "[chore]: Update dependencies to latest versions"
```

   h. **wip:** to mention that the feature being implemented is still incomplete

```
git commit -m "[wip]: Implement feature to run"
```

3. **Line Length:** Always keep your commit message under 72 characters to ensure readability

4. **Use capitalization and punctuation**
   Start the summary with a capital letter and end it with a period. Be consistent in your style
   (e.g., "*Add player movement script.*" or "*Fix collision detection issue.*").

5. **Avoid “and”**
   Try to keep each commit **focused on a single change** and **commit frequently.** Avoid using "and" in commit messages, as it might indicate that the commit is trying to do too much.

Remember that commit messages are a form of documentation for your game project. Clear and well-structured commit messages make it easier for you and your team to understand the history of the project, track changes, and troubleshoot issues effectively. Please follow these naming conventions to help maintain a clean and informative Git history in your game development project.

Git Branch

Sometimes you might be assigned to work on a specific aspect of the game's functionality or to test new features. In such cases, it's better to create and work on your own, dedicated, "branch" to avoid merge conflicts and other complications in the main repository.

How to create your own branch?

i) To create a new branch, use the following command:

```
git checkout -b <new-branch-name>
```

ii) To confirm that the new branch was created, you can use the following command to *list all branches:*

```
git branch
```

iii) If you want to share your new branch with others, you can push it to the remote repository using:

```
git push origin <new-branch-name>
```

Branch Naming Conventions

- **Clear and Consistent**
  Use a clear and consistent branch naming convention to help team members quickly understand the purpose of each branch.

- **Example Branch Names**

```
feature/your-feature-name
bugfix/issue-description
refactor/what-you-refactored
```

Git Pull Requests

When you are working on your own branch, if you feel confident and comfortable with your changes and would like to merge it to the main branch, you can do so with a **pull request (PR).**
Once you submit a PR, your team lead will review it and discuss with you if there are any concerns before approving the merge to the main branch.

Following certain standards when it comes to submitting Pull Requests can really improve and easen the development process. Bad PR's almost always slow down the development process.

Here are some guidelines you must follow before submitting a pull request:

1. **Clear Pull Request Titles and Descriptions**
   Your PR title and description should provide a clear summary of:
   a. The problem being solved
   b. The solution being proposed
   c. The testing strategy (How did you test/ensure your proposed solution)

2. **Direct Communication**
   For resolving disagreements, direct communication via video calls or instant messaging is more efficient than comment threads. After resolving disagreements through direct communication with your team lead, please update the PR thread with the decisions made and things discussed to keep a clear record.

3. **Prompt Feedback**
   As soon as you submit your pull request, please inform your team lead so that they can get back to you within the shortest time possible. This helps in solving issues early on while your code is still 'fresh in your mind', without wasting time on figuring out things again.